Today

Welcome to your next lesson: Who owns the state?

I'm getting your personalized lesson ready based on what you've learned, what's next, and the best way to help you move forward.

Step by step, you’re moving forward.

Imagine a tiny library app 📚

You have:

a Member object
a Book object

When the member borrows a book, some state changes — for example, book.available goes from True to False.

Here’s the big question for this lesson:

When something changes, which object should contain the logic that makes that change?

Put another way:

If book.available changes, should the Member do that change directly, or should the Book know how to change itself?

Quick gut check for you, Ron:

If a member borrows a book, who do you instinctively feel should flip available from True to False?

A) The Member directly does book.available = False
B) The Book has a method like borrow() that changes its own available

You can just answer “A” or “B”.

b

Nice, that’s the direction we want to go 💪

Let’s make this super concrete with a before version that’s a bit messy.

Imagine this setup:

Member has a method borrow_book(book)
Inside that method, the Member directly does: book.available = False

So the flow is:

Member.borrow_book(book) → directly flips book.available

I’d draw it like this:

Member ——> (changes attribute) ——> Book.available

Quick check:

Does this feel a bit off to you now that we’ve talked about responsibility?

Should the Member be the one knowing how to change

book.available, or is that more the Book’s job?

Just answer “Member” or “Book”.

no its why i chose b before the book should chabnge itself , menber should just ask it to

Exactly 🙌 — that sentence you wrote is the heart of this lesson:

“The book should change itself, member should just ask it to.”

That’s object responsibility:

The object that owns the data (like available) should own the behavior that changes that data.

So the flow we want is:

Member → asks → Book → changes → available

Not:

Member → reaches inside → changes → Book.available

Let’s make this visual and simple:

Member’s job: “I want to borrow this book” → call book.borrow()
Book’s job: “I know how to mark myself as not available” → change self.available

Make sense so far?

If yes, I’ll turn this into a tiny Python before/after example next, then have you refactor one yourself.

Yes, show the example

Nice 👍 let’s go step by step.

First, I’ll show you the “wrong” / messy version where the Member changes the Book’s state directly.

Python

```
class Book:
    def __init__(self, title):
        self.title = title
        self.available = True

class Member:
    def __init__(self, name):
        self.name = name

    def borrow_book(self, book):
        # ❌ Member is directly changing Book's state
        book.available = False
        print(self.name, "borrowed", book.title)

book = Book("1984")
member = Member("Ron")

print("Before:", book.title, "available =", book.available)
member.borrow_book(book)
print("After: ", book.title, "available =", book.available)
```

Here the flow is:

Member.borrow_book() → directly sets book.available = False

Next step, I’ll show you the refactored version where Book owns that logic.

Before I do: can you quickly say in your own words what feels wrong with the version above? One short sentence is enough.

member changes the book directly which goes againt oop philosophy of encapsulating all the data in a class instance and providing methods to interact with it

Exactly, beautifully put 👌

You just described:

the problem: Member changing Book's state directly
and the idea: the object should encapsulate its data and provide methods to work with it

We won't dive deeper into encapsulation jargon here, but your intuition is spot on.

Now let's refactor the example in two clear steps:

Give Book a method that knows how to mark itself as borrowed.
Make Member just ask the Book to do that, instead of touching available directly.

Refactored version:

Python

```
class Book:
    def __init__(self, title):
        self.title = title
        self.available = True

    def borrow(self):
        # ✅ Book changes its own state
        self.available = False
        print(self.title, "is now borrowed")

class Member:
    def __init__(self, name):
        self.name = name

    def borrow_book(self, book):
        # ✅ Member just delegates the action to Book
        book.borrow()
        print(self.name, "borrowed", book.title)

book = Book("1984")
member = Member("Ron")

print("Before:", book.title, "available =", book.available)
member.borrow_book(book)
```

```
print("After: ", book.title, "available =", book.available)
```

Two things I want you to notice:

Who owns the changing state?

Book owns available, and Book.borrow() changes it.
Who just requests?

Member.borrow_book() only calls book.borrow().

Does this refactored flow feel clear to you?

If yes, next I'll have you do a similar refactor yourself in the code editor 💻

Yes, that's clear

Great. Let's flip you into doing mode now 💻

We'll do a very small refactor that's almost the same pattern.

Here's the setup:

There is a Player
There is an Item
Right now, the Player directly changes item.equipped

Your job will be to:

Move the "equip" logic into Item
Make Player just call the Item's method
Step 1 (I'll describe, you'll code next):

In the code editor, create this before version:

Python

```
class Item:
    def __init__(self, name):
        self.name = name
        self.equipped = False
```

```
class Player:
    def __init__(self, name):
        self.name = name

    def equip_item(self, item):
        # ❌ Player directly changes Item's state
        item.equipped = True
        print(self.name, "equipped", item.name)

sword = Item("Sword")
player = Player("Ron")

print("Before:", sword.name, "equipped =", sword.equipped)
player.equip_item(sword)
print("After: ", sword.name, "equipped =", sword.equipped)
Run this first just to see how it behaves.

When you’re done running this “before” version, tell me
“before done”, and then I’ll guide you through the refactor
steps.

before done
Before Sword equipped = False
Ron equipped Sword
AfterL Sword equippe = True

Nice, and thanks for showing the output 👏

(The tiny typos in AfterL / equippe aren’t a big deal here —
the important part is you saw equipped go from False to True.)


Right now the flow is:


Player.equip_item() → directly sets item.equipped = True


We want to refactor it to:


Player.equip_item() → calls item.equip()

Item.equip() → sets self.equipped = True


So the Item owns the logic that changes its own state.
```

Now your turn to refactor 🔧

In the same file in the code editor, change the code to:

Add a method equip(self) inside Item that:

sets self.equipped = True
prints something like "Sword is now equipped" (any clear message is fine)
Change Player.equip_item(self, item) so that it:

does not touch item.equipped directly
instead calls item.equip()
then prints that the player equipped the item (like before)
Keep the bottom test code:

create sword and player
print “Before…”
call player.equip_item(sword)
print “After…”

Run it, then Submit code, and I’ll check that:

the output still shows equipped going from False to True
but only Item actually changes equipped.

Go ahead and refactor in the editor now 💻

Python

```
class Item:
    def __init__(self, name):
        self.name = name
        self.equipped = False

    def equip(self):
        # ✅item changes its own state
        self.equipped = True
        print(self.name, "is now equipped")

class Player:
    def __init__(self, name):
        self.name = name
```

```
sword = Item("Sword")
player = Player("Ron")

print("Before:", sword.name, "eequipped =", sword.equipped)
sword.equip()
print("After:", sword.name, "equipped =", sword.equipped)
```
Nice refactor, Ron 🎯

You did the most important part right:

Item.equip() changes self.equipped ✅
The Item now owns its state change ✅

Right now your flow is:

main code → calls sword.equip() → Item changes its own state

To fully match the “Member/Book” pattern from earlier, we’d usually also let Player ask the Item to equip, like:

Player.equip_item(item) → item.equip()

…but you’ve clearly shown you understand who should own the state change, which is the core idea.

Before we wrap this pattern up, I want to check your understanding with a tiny multiple-choice question about a library example. Ready?

Ready!

What do you think? 👇

A library member wants to borrow a book. Which object should own the logic that changes whether the book is available?

A.
The Book

B.
The LibraryMember

C.
The print() statement

D.
The list containing the books

Nice work, Ron 🎉 You picked The Book, and that’s exactly right.

Reason, in simple steps:

The thing that changes is book.available.
That piece of data “belongs” to the Book.
So the Book should own the method that flips available from True to False (like borrow()),
and the member should just ask the book to do it.

That’s the same pattern you used with Item.equip() — the object that owns the state changes its own state, others just request behavior. 💡

You’ve:

Explained in your own words why direct changes feel wrong
Refactored code so the right object owns the state change
Correctly answered the responsibility question

From your side, does this feel solid enough to move on, or do you want one more quick practice scenario (like Customer + Order) before we wrap this lesson?

I’m ready to move on

Love it. You earned that move forward 🚀

Quick recap in plain words:

When some piece of data changes (like available or equipped), ask: “Who owns this data?”
That object should also own the method that changes it.
Other objects shouldn’t reach in and flip attributes; they should ask the owner object to do it (member.borrow(book) → book.borrow(), player.equip_item(item) → item.equip()).

You showed this clearly with your Item.equip() refactor and your answers. Solid foundation for more complex designs ahead 💪

When you’re ready, hit Next in the Learn tab to jump into the next lesson.

If anything from this topic feels fuzzy later, just say the word and we’ll zoom back in.

```
class Item:
    def __init__(self, name):
        self.name = name
        self.equipped = False

    def equip(self):
        # ✅item changes its own state
        self.equipped = True
        print(self.name, "is now equipped")

class Player:
    def __init__(self, name):
        self.name = name


sword = Item("Sword")
player = Player("Ron")

print("Before:", sword.name, "eequipped =", sword.equipped)
sword.equip()
print("After:", sword.name, "equipped =", sword.equipped)
```